

SKILL.state: Scalable Long-Horizon Agent Skills

Sanket Badhe¹, Priyanka Tiwari², Jonghyun Chung¹,

¹Google LLC, ²Purdue University **Correspondence:** sanketbadhe@google.com

Abstract

Large Language Models (LLMs) increasingly act as autonomous agents executing complex, long-running procedural skills. Existing agent runtimes maintain execution by continually appending observations, actions, and intermediate reasoning traces to an ever-growing conversation history, causing latency degradation and context-poisoning failures over long horizons. We present SKILL.state, a runtime architecture that replaces append-only conversational history with an explicit, mutable execution state. At each execution step, the model receives only the immutable skill specification, the current structured execution state, and the latest observation. Intermediate reasoning is discarded immediately after producing a validated state update, preventing prompt growth with execution history. Across diverse datasets, models, and execution environments, SKILL.state improves task accuracy while substantially reducing cumulative token consumption. Our results demonstrate that explicit execution state is an effective and architecture-agnostic abstraction for scalable long-horizon agent skills.

1 Introduction

Large Language Models (LLMs) have rapidly evolved from passive language interfaces into autonomous systems capable of iterative reasoning, tool use, and interaction with external environments (Yao et al., 2022; Schick et al., 2023; Qin et al., 2024; Wu et al., 2023). Recent work further demonstrates that these capabilities can be encapsulated as reusable procedural skills, enabling agents to perform software engineering, workflow automation, web interaction, and scientific discovery through modular compositions of specialized behaviors (Badhe et al., 2026). As agents increasingly execute long-running procedures, execution itself becomes a systems problem rather than purely a reasoning problem.

Modern agent runtimes almost universally adopt a conversational execution model. At every execution step, the language model receives the original skill specification together with an ever-growing transcript of previous reasoning, actions, observations, and tool outputs (Yao et al., 2022; Mialon et al., 2023). Although memory systems alleviate context growth through summarization or retrieval (Packer et al., 2023; Wang et al., 2023; Zhong et al., 2023), they preserve the same execution semantics: future decisions are conditioned on textual reconstructions of past execution rather than an explicit representation of the current execution state.

This design introduces fundamental limitations for long-horizon procedural skills. Prompt size grows with execution length, increasing token consumption and inference cost (Liu et al., 2024; Xiao et al., 2024a). Historical observations and obsolete reasoning remain embedded in the context long after they cease to be relevant, requiring the model to continually distinguish current facts from historical artifacts. Consequently, execution correctness increasingly depends on reconstructing state from accumulated textual history.

In this paper, we introduce SKILL.state, a runtime architecture that reformulates procedural skill execution as explicit state transitions rather than conversational history accumulation. Figure 1 provides an overview of the proposed runtime. At execution step t , the language model receives only three inputs:

$$A_t = (P, \Sigma_t, O_t), \quad (1)$$

where P denotes the immutable procedural specification, Σ_t is the structured execution state, and O_t is the latest environment observation. After producing a validated state update, the intermediate reasoning trace is discarded while only the updated execution state is retained. Consequently, execution depends strictly on the current world state instead of replaying historical trajectories.

To evaluate this hypothesis, we evaluate SKILL.state across both synthetic and real-world benchmarks: SkillExecBench, a controlled benchmark designed for long-horizon procedural skill execution under scaling, noise, and state recovery; InterCode CTF (Yang et al., 2023), featuring interactive Linux terminal exploitation; and Sierra τ -Bench (Yao et al., 2024), evaluating multi-turn customer-service workflows over complex database APIs.

Experimental results demonstrate that explicit execution state substantially improves the scalability of long-horizon procedural skills by maintaining bounded prompt sizes while cutting token consumption and outperforming history-based and compression-based baselines across multiple model families.

Our contributions are summarized as follows:

- We propose SKILL.state, a runtime architecture that executes procedural skills through explicit structured execution state where intermediate reasoning is discarded after each step, proving a strictly bounded $\mathcal{O}(1)$ prompt footprint and $\mathcal{O}(T)$ cumulative token complexity.
- We present SkillExecBench, alongside evaluations on public benchmarks (InterCode CTF and Sierra τ -Bench), for evaluating long-horizon procedural skill execution in sequential, stateful environments.
- Across multiple execution horizons and runtime baselines, we demonstrate that state-centric execution maintains competitive task performance while substantially reducing prompt growth and cumulative token consumption across both proprietary and open-weight models.

2 Related Work

2.1 Procedural Skills for LLM Agents

Existing research on reusable procedural skills primarily addresses skill discovery, representation, composition and and security threat modeling (Badhe et al., 2026; Badhe and Tiwari, 2026). Our work instead focuses on the largely unexplored mechanics of skill execution once a skill has been selected.

2.2 Memory Architectures for Long-Horizon Agents

Long-horizon agent architectures typically preserve conversational semantics through episodic retrieval (Park et al., 2023) or persistent storage (Chhikara et al., 2025; Zhong et al., 2024). These methods leave execution state implicitly distributed across accumulated logs. SKILL.state instead isolates execution into an explicit, mutable runtime state, eliminating the need to repeatedly reconstruct world models from textual history. Frameworks like LangGraph use auxiliary structured state to orchestrate workflows across agent nodes. However, these systems still rely on conversational transcripts as the primary reasoning substrate. SKILL.state replaces this substrate by discarding intermediate reasoning traces immediately after producing validated state transitions.

2.3 Dialogue State Tracking

Dialogue State Tracking (DST) maintains user slot values across conversational turns in task-oriented dialogue (Williams et al., 2013; Henderson et al., 2014; Rastogi et al., 2020; Wu et al., 2019; Heck et al., 2020; Hosseini-Asl et al., 2020). While both DST and SKILL.state maintain structured representations, they differ fundamentally in execution mechanics: DST tracks auxiliary state alongside full conversational transcripts in quasi-static dialogues, whereas SKILL.state treats the structured state as a sufficient statistic, discarding conversational history to execute autonomous skills in dynamic environments with bounded prompt footprints.

2.4 Context Management and Long-Context Reasoning

Language models exhibit degraded retrieval over long contexts (Liu et al., 2024; Zhang et al., 2024), motivating streaming attention (Xiao et al., 2024b) and prompt compression techniques (Jiang et al., 2023; Li et al., 2023). Rather than attempting to process or compress extended conversational histories, SKILL.state prevents history accumulation entirely by maintaining the canonical execution state required for the next computation.

3 SKILL.state

Current LLM agent runtimes execute procedural skills by repeatedly appending reasoning traces, actions, observations, and tool outputs to a growing conversational history. Consequently, the execution

state is represented implicitly within natural language and must be reconstructed by the language model at every interaction. As execution horizons increase, both prompt size and the volume of obsolete information grow monotonically, making execution increasingly dependent on interpreting historical text rather than maintaining the current world state.

SKILL.state reformulates procedural skill execution as an explicit state transition process. Instead of representing execution as an append-only conversation, every execution step is defined by:

$$A_t = (P, \Sigma_t, O_t), \quad (2)$$

where P is the immutable procedural specification, Σ_t is the structured execution state at step t , and O_t is the latest observation received from the environment. The language model never receives previous observations, previous actions, or previous reasoning traces.

Figure 1 illustrates the execution cycle. At each step, the runtime constructs a prompt from (P, Σ_t, O_t) , invokes the language model, deterministically validates the proposed state transition, updates the execution state, executes the selected action, and repeats the process using the updated state.

3.1 Execution State and Schema Authoring

Unlike conversational runtimes, SKILL.state treats execution state as a first-class runtime abstraction. The state contains only information required for future execution and is represented using a structured schema defined for the domain. Schemas are authored once per domain rather than per task; for example, across all 100 diverse challenge instances in the InterCode CTF benchmark, the agent reuses a single static 5-field schema (discovered_flags, tested_hypotheses, active_files, working_dir, cmd_summary).

3.2 Reasoning and State Transitions

Reasoning is used strictly as an intermediate computation for producing state transitions and selecting the next action. Given the current execution context (P, Σ_t, O_t) , the language model generates:

$$(R_t, \Delta\Sigma_t, a_t), \quad (3)$$

where R_t denotes the multi-step Chain-of-Thought reasoning trace, $\Delta\Sigma_t$ is a structured state update (a

Algorithm 1 SKILL.state Runtime

Require: Procedural specification P , initial state Σ_0

```

1: for  $t = 0, \dots, T$  do
2:   Receive latest observation  $O_t$ 
3:   Construct prompt  $(P, \Sigma_t, O_t)$ 
4:   Generate  $(R_t, \Delta\Sigma_t, a_t)$  using the LLM
5:   Validate  $\Delta\Sigma_t$ 
6:    $\Sigma_{t+1} \leftarrow \Sigma_t \oplus \Delta\Sigma_t$ 
7:   Execute  $a_t$ 
8: end for
```

JSON dictionary of key mutations and deletions), and a_t is the action to execute.

Crucially, within-step multi-step reasoning is fully intact during generation to support complex deductive planning. However, once the state transition has been validated and applied, the reasoning trace R_t is discarded permanently and never appears in subsequent prompts. The execution state is updated according to:

$$\Sigma_{t+1} = \Sigma_t \oplus \Delta\Sigma_t, \quad (4)$$

where $\oplus$ denotes the runtime’s dictionary merge operator with null-deletion semantics. This model projects transient reasoning into persistent structured state, allowing only information required for future execution to survive across interactions.

Algorithm 1 summarizes the execution process.

3.3 Complexity Analysis

Let T denote the execution horizon. For conversational runtimes, prompt length grows with the accumulated interaction history, $|C_t| = \mathcal{O}(t)$, leading to cumulative token complexity:

$$\sum_{t=1}^T |C_t| = \mathcal{O}(T^2). \quad (5)$$

In contrast, SKILL.state maintains only the procedural specification, structured execution state, and latest observation:

$$|P_t| = \mathcal{O}(|P| + |\Sigma| + |O|), \quad (6)$$

which is asymptotically bounded and independent of the number of previously executed turns t . Consequently, cumulative prompt complexity grows strictly linearly with the execution horizon:

$$\sum_{t=1}^T |P_t| = \mathcal{O}(T). \quad (7)$$

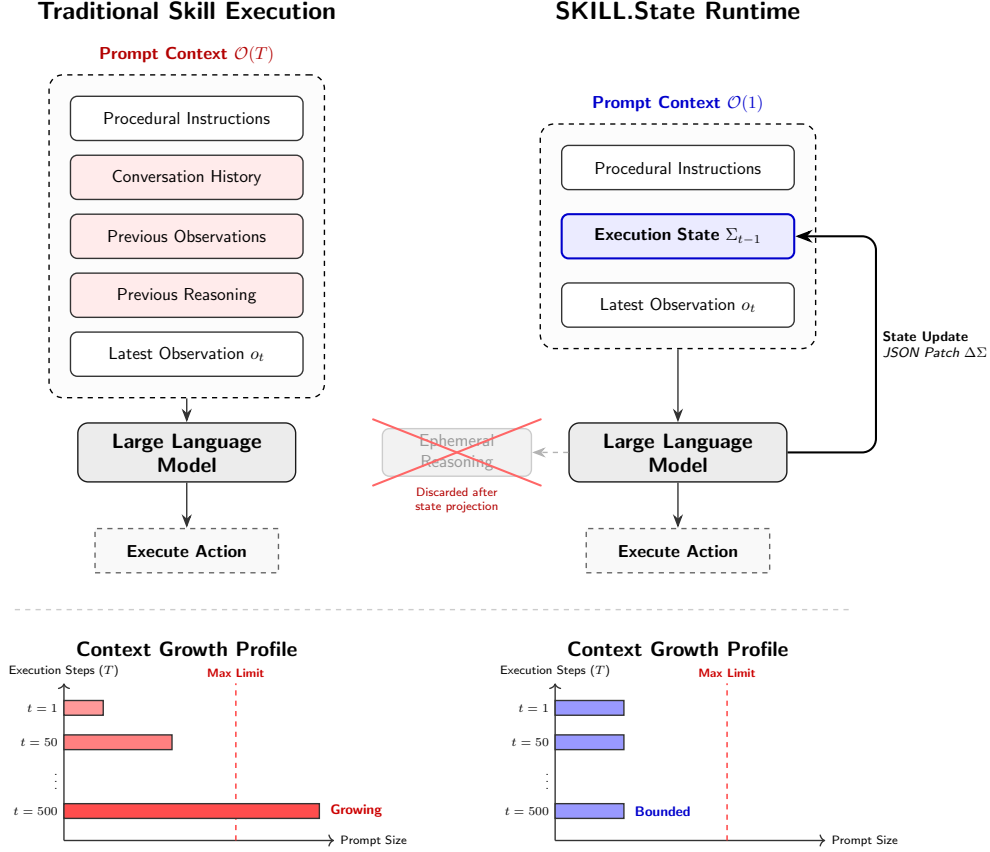


Figure 1: Overview of the SKILL.state architecture.

The resulting runtime shifts execution from reconstructing history toward maintaining an explicit, validated representation of the current execution state.

4 Evaluation Benchmarks

4.1 SkillExecBench (Controlled Diagnostic Testbed)

SkillExecBench isolates execution mechanics from open-ended heuristic search by providing sequential procedural tasks with deterministic ground-truth world transitions:

- **Environment 1 (Warehouse Management):** A discrete physical inventory domain tracking 500 independent shelves. Actions include Store, Ship, Move, and Wait. This environment tests the model’s ability to maintain independent, non-overlapping state variables over extended horizons where early observations leave the context window.
- **Environment 2 (Software Repository):** A deeply nested, relational graph of Git branches,

commits, Pull Requests, and CI test statuses. Actions include CherryPick, Merge, RunTests, CreateRelease, and Rollback. Features dense dependencies where a single action (e.g., merging a PR) fundamentally alters the state of the target branch and dependent PRs, testing complex structural reasoning over an entangled graph.

4.2 Public Interactive Benchmarks

To evaluate SKILL.state on real-world, non-deterministic tasks with complex search, generation, and tool use, we evaluate on two public benchmarks:

- **InterCode CTF (Yang et al., 2023):** A suite of 100 Linux bash Capture-The-Flag challenges spanning reverse engineering, forensics, cryptography, and binary exploitation. Agents execute bash commands in Docker containers and iteratively test hypotheses to discover hidden flags.
- **Sierra τ -Bench (Yao et al., 2024):** A benchmark for tool-agent-user interaction in enterprise customer service (Retail and Airline domains).

Agents interact with simulated users, query relational SQLite databases via tool calls, and execute transactional actions (e.g., flight rebooking, refunds) under business policy constraints.

4.3 Evaluation Metrics

We evaluate runtimes across three dimensions:

- **Task Accuracy / Success Rate:** In SkillExecBench, accuracy is measured continuously as the ratio of valid, correct actions matching the ground-truth deterministic simulation ($\text{Score} = \frac{\text{Successful Actions}}{\text{Total Actionable Events}}$). In InterCode CTF, success is binary pass@1 (exact match on the binary-verified flag). In τ -Bench, success is scored by the official programmatic evaluator, which verifies that the final database state satisfies user intent without policy violations.
- **Average Prompt Size:** The mean string character length per LLM invocation.
- **Total Token Cost:** The cumulative token burn across the entire execution horizon.

5 Experiments and Results

5.1 Experimental Setup

We evaluate SKILL.state against two families of baselines (see Appendix 7 for exact prompt templates):

Primary Runtime Paradigms:

1. **Prompt (ReAct-style):** Appends every observation, intermediate reasoning trace, and action to a continually growing transcript (Yao et al., 2022).
2. **Memory (Summarization-style):** Maintains a rolling 3-step conversational window alongside a periodically updated natural language summary of past interactions.
3. **Stateful (LangGraph-style):** Injects a structured state block into the context window alongside the full rolling conversational transcript.

Budget-Matched and Compression Controls:

1. **Truncated (Sliding Window):** Retains only the most recent interaction turns that fit within a fixed token budget.
2. **Summary-capped:** Strictly enforces a hard token ceiling on the natural language summary.

3. **ReAct + LLMingua (Jiang et al., 2023):**

Uses budget-aware small-model perplexity compression to prune tokens from the full history down to the target budget.

Underlying Models: Evaluations are conducted across proprietary and open-weight models: Gemini-3-Flash, Gemma-4-31B-it, and Qwen-3-8B-it. Decoding is controlled at temperature 0.0 and top- p 1.0 across all runs to ensure deterministic reproducibility.

Statistical Significance: All synthetic experiments are evaluated across 5 distinct procedural generator seeds. Results are reported as mean $\pm$ sample standard deviation. Differences between SKILL.state and baselines at extended horizons ($T \geq 50$) are statistically significant (paired t -test, $p < 0.01$).

5.2 Experiment 1: Long-Horizon Execution Scaling

We evaluate runtime accuracy and context expansion across execution horizons scaling from $T = 10$ to $T = 200$ steps.

Results: As shown in Table 1, SKILL.state matches or exceeds baseline accuracy across all horizons while maintaining a flat prompt size. In contrast, history-appending baselines suffer quadratic token accumulation $\mathcal{O}(T^2)$. At $T = 100$, the Stateful baseline consumes 1,062,387 tokens, whereas SKILL.state consumes only 65,408 tokens (a $16.2\times$ token reduction). At $T = 200$, SKILL.state maintains 0.94 accuracy consuming 122k tokens, while the Memory baseline inflates to 6.1M tokens. Additional scaling results for the Software Repository and open-weight models are detailed in Appendix 7.

5.3 Experiment 2: Context Corruption (Noise Robustness)

Real-world execution environments emit dense background telemetry. We fix the horizon at $T = 50$ and inject distractor events (system telemetry, irrelevant git branch activities, and rule overrides) at rates of 5, 20, and 50 events per turn (see Appendix 7 for calibration details).

Results: As shown in Table 2, the standard Prompt runtime degrades sharply from 0.68 at low noise down to 0.53 at high noise. In contrast, SKILL.state maintains robust task completion (≥ 0.97) across all noise levels because distractors are filtered out during state patch generation and never enter subsequent prompts.

Table 1: Warehouse Management Long-Horizon Scaling using Gemini-3-Flash. Baseline runtimes suffer $\mathcal{O}(T^2)$ context accumulation, whereas SKILL.state maintains a bounded $\mathcal{O}(1)$ prompt footprint (Mean $\pm$ SD across 5 seeds).

Horizon (T)	Runtime	Score (Accuracy)	Avg Prompt (Chars)	Total Tokens
10	Prompt (ReAct)	0.90 ± 0.02	3,249 ± 94	9,438 ± 371
	Memory (Summary)	1.00 ± 0.00	3,300 ± 123	9,972 ± 204
	Stateful (LangGraph)	1.00 ± 0.00	3,430 ± 42	10,337 ± 299
	SKILL.state	1.00 ± 0.00	1,775 ± 74	5,870 ± 131
25	Prompt (ReAct)	0.92 ± 0.02	6,052 ± 192	42,689 $\pm 2,238$
	Memory (Summary)	0.99 ± 0.00	6,357 ± 203	43,067 $\pm 1,948$
	Stateful (LangGraph)	1.00 ± 0.00	5,858 ± 301	41,238 $\pm 3,196$
	SKILL.state	1.00 ± 0.00	1,736 ± 49	14,714 ± 564
50	Prompt (ReAct)	0.88 ± 0.04	11,931 ± 346	171,658 $\pm 6,978$
	Memory (Summary)	0.93 ± 0.03	7,582 ± 283	131,455 $\pm 6,841$
	Stateful (LangGraph)	0.94 ± 0.00	11,594 ± 438	170,992 $\pm 7,918$
	SKILL.state	0.96 ± 0.01	1,773 ± 53	30,151 $\pm 1,231$
100	Prompt (ReAct)	0.84 ± 0.07	36,362 $\pm 1,304$	1,245,413 $\pm 53,241$
	Memory (Summary)	0.87 ± 0.05	29,607 ± 978	1,082,154 $\pm 83,212$
	Stateful (LangGraph)	0.91 ± 0.02	31,354 ± 831	1,062,387 $\pm 53,839$
	SKILL.state	0.94 ± 0.01	1,905 ± 93	65,408 $\pm 5,431$
200	Prompt (ReAct)	0.74 ± 0.14	48,007 $\pm 2,092$	2,608,755 $\pm 102,415$
	Memory (Summary)	0.84 ± 0.09	84,364 $\pm 3,446$	6,175,509 $\pm 294,089$
	Stateful (LangGraph)	0.88 ± 0.03	72,305 $\pm 3,096$	5,041,164 $\pm 346,925$
	SKILL.state	0.94 ± 0.02	1,811 ± 184	122,384 $\pm 4,522$

Table 2: Warehouse Noise Robustness ($T = 50$, Gemini-3-Flash).

Noise Level	Runtime	Score
5 Events (Low)	Prompt	0.68
	Memory	1.00
	Stateful	1.00
	SKILL.state	1.00
20 Events (Medium)	Prompt	0.61
	Memory	1.00
	Stateful	0.98
	SKILL.state	0.97
50 Events (High)	Prompt	0.53
	Memory	0.96
	Stateful	0.98
	SKILL.state	0.98

5.4 Experiment 3: State Recovery

We test runtime resilience to silent external environment drift where the true world state is modified outside the agent’s action loop (e.g., an external actor moves an inventory item).

Results: As shown in Table 3, history-based baselines hallucinate for 5 to 8 consecutive turns because obsolete facts in their prompt history over-

Table 3: Warehouse State Recovery (Gemini-3-Flash).

Scenario	Runtime	Success	Recovery Steps
A: Secret Audit	Prompt / Memory / Stateful	Yes	5–8
	SKILL.state	Yes	0
B: Secret Barcode	Prompt / Memory / Stateful	Yes	6–8
	SKILL.state	Yes	0
C: Secret Move	Prompt / Memory / Stateful	Yes	5–8
	SKILL.state	Yes	0
D: Canceled Order	All Runtimes	No	N/A

power contradictory new observations. In sharp contrast, SKILL.state requires zero recovery steps: because its decisions depend on the current structured state, the state is updated immediately upon receiving the corrective alert.

5.5 Experiment 4: Public Interactive Benchmarks

To test generalizability on open-ended tasks with complex search, generation, and tool use, we evaluate SKILL.state on InterCode CTF and Sierra τ -Bench.

Results: As shown in Table 4, SKILL.state achieves the highest task completion rates across all

Table 4: Evaluation on Public Interactive Benchmarks using Gemini-3-Flash. SKILL.state achieves the highest task success rates while significantly reducing prompt sizes and cumulative token consumption.

Runtime	InterCode CTF (100 Tasks)			Sierra τ -Bench (Retail)			Sierra τ -Bench (Airline)		
	Pass@1	Prompt	Tokens	Pass Rate	Prompt	Tokens	Pass Rate	Prompt	Tokens
Prompt (ReAct)	43.2%	1,909	977k	48.2%	2,819	4.48M	21.8%	5,100	4.85M
Memory (Summary)	46.4%	1,797	1.03M	29.9%	2,737	4.24M	23.6%	4,700	4.65M
Stateful (LangGraph)	41.8%	1,946	1.13M	51.7%	3,065	3.92M	28.1%	5,400	5.28M
SKILL.state	54.2%	813	387k	58.3%	3,325	3.47M	32.4%	2,800	2.88M

three benchmarks while substantially cutting cumulative token consumption. In InterCode CTF, maintaining explicit hypotheses and discovered flags in Σ_t prevents the model from repeating failed commands, increasing pass@1 to 54.2% (+7.8 points over the strongest baseline and +12.4 points over Stateful) while cutting total tokens by 60.4% vs. ReAct and 65.9% vs. Stateful. In τ -Bench Retail, SKILL.state leads with 58.3% pass rate at the lowest total token cost. In τ -Bench Airline, where complex database responses cause baseline prompts to peak above 11,000 tokens/step, SKILL.state maintains a flat footprint of $\sim 2,800$ tokens/step and achieves a 32.4% pass rate, saving 40.5% tokens vs. ReAct and 45.4% vs. Stateful.

Table 5: Budget-Matched Controls on Warehouse ($T = 100$, Gemini-3-Flash, Budget $\sim 1,800$ tokens).

Runtime / Configuration	Score	Avg Prompt	Total Tokens
Full ReAct (Unbounded)	0.84	36,362	1,245,413
Truncated (Sliding Window)	0.18	1,800	62,100
Summary-capped	0.52	1,840	63,400
ReAct + LLMingua	0.22	1,810	62,350
SKILL.state (Structured)	0.94	1,905	65,408

5.6 Experiment 5: Budget-Matched Controls and Statistical Compression

To determine whether SKILL.state’s performance gains stem merely from shorter prompts or from structured state representation, we evaluate budget-matched baselines on Warehouse ($T = 100$, Gemini-3-Flash) pinned to the token budget of SKILL.state ($\sim 1,800$ characters). Full multi-horizon scaling across $T \in \{10, 25, 50, 100\}$ is detailed in Appendix .2.

Results: As shown in Table 5, all budget-matched compression baselines suffer catastrophic failure. Sliding-window truncation drops to 0.18 because critical early inventory allocations are evicted. LLMingua drops to 0.22 because statistical entropy filtering removes seemingly redundant slot identifiers that are semantically vital. In contrast,

SKILL.state achieves 0.94 score, demonstrating that structured state maintenance preserves exact relational dependencies that statistical compressors destroy.

5.7 Error Taxonomy for Open-Weight Models

On open-weight models (Gemma-4-31B at $T = 100$, score 0.42), we analyze failure logs and categorize errors into three distinct modes:

- 1. Premature State Overwrite / Deletion (68%):** The model accidentally omits existing keys during state update rather than merging in-place.
- 2. Schema Comprehension / Type Coercion (20%):** Inconsistencies between expected nested lists and dictionaries.
- 3. JSON Syntax / Formatting Slips (12%):** Malformed JSON delimiters or trailing commas.

This error distribution shows that small-model degradation stems from structured output adherence rather than reasoning capacity, motivating constrained decoding in future runtime iterations.

6 Conclusion

We presented **SKILL.state**, a runtime architecture that replaces append-only conversational history with explicit, structured execution state. By discarding intermediate reasoning traces after each validated transition, SKILL.state maintains a bounded $\mathcal{O}(1)$ prompt footprint and scales linearly $\mathcal{O}(T)$ in cumulative tokens. Across controlled diagnostic tasks and public interactive benchmarks, explicit execution state consistently improves task accuracy while substantially reducing prompt growth and token consumption.

7 Limitations

SKILL.state assumes that the execution state can be made a *sufficient statistic* for future execution: that everything in the past bearing on future actions

can be projected into the structured state as soon as it becomes known. Where this holds, discarding intermediate reasoning and conversational history is lossless. However, this assumption fails in three distinct settings: (1) when no fixed schema is known in advance and the relevant state structure must be discovered dynamically during execution; (2) when a correct state update depends on an earlier observation whose relevance was not recognized when first observed, and was therefore never committed to state; and (3) when the task objective is defined over the historical trajectory itself (e.g., auditing, debugging provenance, or explaining past actions), where interaction history is the target output rather than operational overhead.

Our current implementation focuses on single-agent procedural execution. While the explicit state abstraction extends naturally to multi-agent systems—where a shared execution state acts as the central coordination substrate instead of exchanging quadratic conversational transcripts—multi-agent environments introduce concurrent writes, requiring deterministic conflict-resolution semantics in the merge operator $\oplus$ that our single-agent setting does not exercise.

Finally, SKILL.state relies on the language model to propose valid structured state patches. Because schema ownership and validation reside in the deterministic runtime rather than the model, malformed outputs cannot corrupt persistent state Σ_t ; an invalid patch triggers a rollback-retry cycle. For smaller open-weight models, integrating grammar-constrained decoding can eliminate syntactic formatting errors, allowing the model to focus entirely on semantic state transitions.

References

- Sanket Badhe, Deep Shah, Priyanka Tiwari, and Nehal Kathrotia. 2026. A systematic survey of agent skills: Lifecycle, taxonomy, and security. *Taxonomy, and Security (July 31, 2026)*.
- Sanket Badhe and Priyanka Tiwari. 2026. [Agent skill security: Threat models, attacks, defenses, and evaluation](#). *Preprint*, arXiv:2607.13987.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.
- Michael Heck, Carel van Niekerk, Nurul Lubis, Christian Geishausen, Hsien-chin Lin, Marco Moresi, and Milica Gašić. 2020. Trippy: A triple copy strategy for value-independent neural dialog state tracking. In *Proceedings of the 21th Annual Meeting of the Special Interest Group on Discourse and Dialogue*, pages 35–44.
- Matthew Henderson, Blaise Thomson, and Jason D Williams. 2014. The second dialog state tracking challenge. In *Proceedings of the 15th Annual Meeting of the Special Interest Group on Discourse and Dialogue (SIGDIAL)*, pages 263–272.
- Ehsan Hosseini-Asl, Bryan McCann, Chien-Sheng Wu, Semih Yavuz, and Richard Socher. 2020. A simple language model for task-oriented dialogue. *Advances in Neural Information Processing Systems*, 33:20179–20191.
- Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 13358–13376.
- Yucheng Li, Bo Dong, Frank Zhang, Dong Wang, Yanzhao Xu, Xinyu Chen, and Xiang Ren. 2023. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6342–6353.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. [Lost in the middle: How language models use long contexts](#). *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Grégoire Mialon, Roberto Dessi, Maria Lomeli, Christoforos Nalmpantis, Ramakanth Pasunuru, Roberta Raileanu, Baptiste Roziere, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, Edouard Grave, Yann LeCun, and Thomas Scialom. 2023. [Augmented language models: a survey](#). *Transactions on Machine Learning Research*. Survey Certification.
- Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. 2023. Memgpt: towards llms as operating systems.
- Joon Sung Park, Joseph C O’Brien, Carrie J Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2024. ToolLLM: Facilitating large language models to master 16000+ real-world apis. In *International Conference on Learning Representations (ICLR)*.
- Abhinav Rastogi, Xiaoxue Zang, Srinivas Sunkara, Raghav Gupta, and Pranav Khaitan. 2020. Towards scalable multi-domain conversational agents: The

- schema-guided dialogue dataset. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 8689–8696.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Weizhi Wang, Li Dong, Hao Cheng, Xiaodong Liu, Xifeng Yan, Jianfeng Gao, and Furu Wei. 2023. Augmenting language models with long-term memory. *Advances in Neural Information Processing Systems (NeurIPS)*.
- Jason Williams, Antoine Raux, Deepak Ramachandran, and Alan Black. 2013. The dialog state tracking challenge. In *Proceedings of the SIGDIAL 2013 Conference*, pages 404–413.
- Chien-Sheng Wu, Andrea Madotto, Ehsan Hosseini-Asl, Caiming Xiong, Richard Socher, and Pascale Fung. 2019. Transferable multi-domain state generator for task-oriented dialogue systems. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 808–819.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, and 1 others. 2023. AutoGen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024a. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024b. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, volume 2024, pages 21875–21895.
- John Yang, Akshara Prabhakar, Karthik Narasimhan, and Shunyu Yao. 2023. InterCode: Standardizing and benchmarking interactive coding with execution feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Shunyu Yao, Noah Shinn, Jeffrey Zhao, Qingyun Wu, and Karthik Narasimhan. 2024. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.
- Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Hao, Xu Han, Zhen Thai, Shuo Wang, Zhiyuan Liu, and Maosong Sun. 2024. [\$\infty\$ Bench: Extending long context evaluation beyond 100K tokens](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15262–15277, Bangkok, Thailand. Association for Computational Linguistics.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye Wang, and Yankai Lin. 2023. MemoryBank: Enhancing large language models with long-term memory. *arXiv preprint arXiv:2305.10250*.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 19724–19731.

Appendix A. Runtime Prompts

This appendix provides the exact system prompts used by the four evaluated runtimes. To ensure reproducibility, all prompts are presented exactly as they were dynamically constructed and formatted in the benchmark execution loop.

A.1 Prompt Runtime (ReAct-style)

```
Instructions:
{skill.instructions}

History:
Observation: {history[0].observation}
Reasoning & Action: {history[0].response}
[... Appends all previous observations and actions ...]

Latest Observation: {observation}
Generate your next reasoning and action (format 'Action: <cmd>'):
```

A.2 Memory-Augmented Runtime

```
Instructions:
{skill.instructions}

Summarized History:
{summary_string_of_past_steps}

Recent History:
Observation: {recent_observations[0]}
Response: {recent_responses[0]}
[... Appends the 3 most recent turns ...]

Latest Observation: {observation}
Generate your next reasoning and action (format 'Action: <cmd>'):
```

A.3 Stateful Runtime (LangGraph-style)

```
Instructions:
{skill.instructions}

Current State:
{json.dumps(state, indent=2)}

History:
Observation: {history[0].observation}
Response: {history[0].response}
[... Appends all previous observations and actions ...]

Latest Observation: {observation}
Update the state if necessary, provide reasoning, and output 'Action: <cmd>'.
To update state, use the format: StateUpdate: {"key": "value"}
```

A.4 SKILL.state Runtime

```
Instructions:
{skill.instructions}

Skill Execution State:
```json
{json.dumps(state, separators=(',', ' '))}
Latest Observation: {observation}

Provide your response with:

1. Step-by-step reasoning (will be discarded after execution)
2. A JSON block fenced with json ... containing both your State Patch and your Action. The JSON block
 MUST have exactly these two keys: { "state_patch": { <dict: your state updates, set keys to
 null to delete> }, "action": "<string: the exact command you want to execute>" }
```

*Note:* The `skill.instructions` placeholder dynamically injects the task-specific system prompt (e.g., the agent’s persona, the available action space, and the environment rules). This ensures that across all runtime evaluations, the agent receives the exact same baseline instructions, isolating context management as the only independent variable.

## Appendix B. SkillExecBench Implementation Details

To maintain focus on the core experimental findings in Section 5, we provide the full implementation details of the SkillExecBench environments, task generation logic, and episode trajectories in this appendix.

### B.1 Environment Design

#### Environment 1: Warehouse Management

- **State Representation:** A discrete inventory mapping of 500 independent shelves (e.g., `shelf_0` through `shelf_499`), where each shelf holds exactly one item string identifier or is null.
- **Action Space:**
  - Store `<item_id> <empty_shelf_id>`
  - Ship `<item_id> <shelf_id>`
  - Move `<item_id> <old_shelf_id> <new_shelf_id>`
  - Wait
- **Observation Format:** Textual alerts triggered by system events, including: *Shipment arrived containing [item]*, *Customer ordered [item]*, and *Maintenance required on [shelf]*.
- **Transition Rules:** If an agent calls Store, the environment validates the shelf is empty before placing the item. If Ship is called, the item is destroyed. Invalid actions (e.g., storing an item on an occupied shelf) return a local error observation and reject the state transition.
- **Success Criterion:** The ratio of successfully executed valid actions matching the ground-truth deterministic simulation ( $\text{Score} = \text{Successful Actions} / \text{Total Actionable Events}$ ).

#### Environment 2: Software Repository

- **State Representation:** A simulated Git repository tracking branch histories, file contents, active Pull Requests (PRs), and Continuous Integration (CI) test statuses.
- **Action Space:** `Commit(branch, file)`, `CreatePR(branch)`, `Merge(pr_id)`, `FixCI(branch)`, `Wait`.
- **Observation Format:** CI/CD webhook notifications (e.g., *CI Pipeline Failed for PR #3*), code review comments, and issue assignments.
- **Transition Rules:** Pushing a commit triggers a background CI evaluation transition. Merging a PR successfully transitions the master branch state and deletes the feature branch.
- **Success Criterion:** The percentage of correctly resolved feature requests merged into master without breaking CI checks.

### B.2 Task Generation

Tasks are procedurally generated using a deterministic random number generator. By seeding the generator, we guarantee that all evaluated baseline runtimes experience the exact same sequence of events, ensuring fair comparisons. The horizon length dictates the number of sequential events the agent must handle.

---

**Algorithm 2** Warehouse Task Generation Pseudo-code

---

```
1: $seed \leftarrow 42$
2: $rng \leftarrow \text{Random}(seed)$
3: $gt_shelves \leftarrow \{\text{shelf_0: Null, ..., shelf_499: Null}\}$
4: $events \leftarrow []$
5: for $i = 1$ to $Horizon$ do
6: $available \leftarrow \text{FindEmptyShelves}(gt_shelves)$
7: $occupied \leftarrow \text{FindOccupiedShelves}(gt_shelves)$
8: $possible_events \leftarrow [\text{Receive}]$
9: if $occupied$ is not empty then
10: $possible_events.append(\text{Order, Maintenance})$
11: end if
12: $event_type \leftarrow rng.choice(possible_events)$
13: $events.append(\text{ConstructObservation}(event_type))$
14: $\text{UpdateGroundTruth}(gt_shelves, event_type)$
15: end for
16: return $events$
```

---

### B.3 Example Episode Trajectory

To illustrate the execution flow, below is a concrete, single-turn trajectory for the **SKILL.state** runtime operating in the Warehouse environment.

#### 1. Observation Received (Environment $\rightarrow$ LLM)

Latest Observation:  
Customer ordered item\_12.

#### 2. LLM Processing (LLM Internal)

Reasoning:  
The customer ordered item\_12. I need to ship it.  
I will check my Skill Execution State to locate it.  
Looking at the state schema, item\_12 is located on shelf\_42.  
I must generate a Ship action and update my state to remove it from shelf\_42.

#### 3. State Update and Action Generation (LLM $\rightarrow$ Runtime)

```
{
 "state_patch": {
 "inventory": {
 "shelf_42": null
 }
 },
 "action": "Ship item_12 shelf_42"
}
```

#### 4. Next Observation (Environment $\rightarrow$ LLM)

Latest Observation:  
Success: Shipped item\_12 from shelf\_42.

## Appendix C. Noise Construction (Experiment 2)

In Experiment 2, we evaluate the runtimes' resilience to dense, irrelevant contextual noise. Real-world systems rarely provide clean, perfectly isolated observation spaces; agents must constantly filter out background telemetry, sensor logs, and system chatter to execute their instructions.

To isolate the problem of *Attention Drag*, the experiments in this paper exclusively focus on **Condition 1: Irrelevant Context**.

## C.1 Noise Properties

For Condition 1 evaluations across both environments, the injected noise strings are defined by three strict properties:

1. **Randomly Generated:** Values such as battery percentages, temperatures, server IDs, and CPU loads are sampled uniformly at random during each execution step.
2. **Strictly Irrelevant:** The semantic meaning of the noise has absolutely no bearing on the agent's primary task (e.g., fulfilling warehouse orders or fixing CI pipelines).
3. **Non-State-Altering:** The noise events never actually change the underlying ground-truth world state. They are purely observational distractors appended to the environment's response payload under a – BACKGROUND TELEMETRY – header.

## C.2 Environment 1 (Warehouse) Distractors

To simulate a realistic noisy warehouse, the generator randomly selects from the following categories to inject irrelevant strings into the agent's observation space:

### 1. Robot Telemetry Logs

Simulates continuous pinging from automated warehouse robots navigating the floor.

Battery: 85%, Temperature: 45C, CPU Load: 72%,  
Speed: 1.2 m/s, Nav Confidence: 95.4%

### 2. Environmental Sensor Logs

Simulates passive HVAC and ambient sensor readings.

[Sensor] Humidity: 45%, Temp: 22.3C, Light: 310 lux, CO2: 450 ppm

### 3. Camera OCR / Vision Logs

Simulates background security camera or computer-vision object detection events.

[Camera OCR] Forklift parked.  
[Camera OCR] Worker entered Zone A.  
[Camera OCR] Safety Vest Detected.

## C.3 Environment 2 (Software Repository) Distractors

To simulate a noisy, enterprise-scale software engineering environment, the generator continuously injects irrelevant cloud infrastructure syslog telemetry into the agent's terminal observations.

### Syslog Telemetry

Simulates passive health-checks and CPU load warnings from disconnected remote servers running in the background.

[Syslog] Server-42 CPU load: 88%, RAM usage: 71%  
[Syslog] Server-17 CPU load: 12%, RAM usage: 45%  
[Syslog] Server-91 CPU load: 99%, RAM usage: 89%

### Example Corrupted Observation (Software, 3 Events):

Latest Observation:  
CI Pipeline Failed for PR #3. Linter error on line 42.

--- BACKGROUND TELEMETRY ---  
[Syslog] Server-42 CPU load: 88%, RAM usage: 71%  
[Syslog] Server-17 CPU load: 12%, RAM usage: 45%  
[Syslog] Server-91 CPU load: 99%, RAM usage: 89%



Table 6: Software Repository Long-Horizon Execution Scaling using Gemini-3-Flash. Baseline runtimes suffer catastrophic  $O(N^2)$  context collapse, whereas SKILL.state maintains an  $O(1)$  prompt footprint.

Horizon	Runtime	Score	Avg Prompt Size	Total Tokens Consumed
<b>10</b>	Prompt	0.89 $\pm 0.11$	3,411 $\pm 197$	11,670 $\pm 841$
	Memory	0.93 $\pm 0.09$	4,379 $\pm 234$	15,732 $\pm 562$
	Stateful	1.00 $\pm 0.00$	4,200 $\pm 321$	14,120 $\pm 318$
	<b>SKILL.state</b>	<b>1.00 <math>\pm 0.00</math></b>	<b>2,298 <math>\pm 134</math></b>	<b>7,608 <math>\pm 149</math></b>
<b>25</b>	Prompt	0.84 $\pm 0.05$	11,754 $\pm 608$	111,970 $\pm 3,314$
	Memory	0.89 $\pm 0.07$	9,399 $\pm 317$	94,629 $\pm 2,839$
	Stateful	0.94 $\pm 0.03$	14,016 $\pm 586$	128,702 $\pm 3,863$
	<b>SKILL.state</b>	<b>0.88 <math>\pm 0.08</math></b>	<b>2,545 <math>\pm 556</math></b>	<b>21,920 <math>\pm 431</math></b>
<b>50</b>	Prompt	0.71 $\pm 0.14$	23,136 $\pm 911$	462,118 $\pm 13,764$
	Memory	0.65 $\pm 0.12$	35,550 $\pm 2,412$	688,182 $\pm 23,539$
	Stateful	0.74 $\pm 0.08$	31,166 $\pm 3,231$	577,027 $\pm 27,293$
	<b>SKILL.state</b>	<b>0.86 <math>\pm 0.04</math></b>	<b>2,545 <math>\pm 63</math></b>	<b>45,100 <math>\pm 894</math></b>
<b>100</b>	Prompt	0.53 $\pm 0.16$	46,270 $\pm 1,847$	1,848,500 $\pm 55,391$
	Memory	0.57 $\pm 0.05$	71,100 $\pm 5,836$	2,752,700 $\pm 82,467$
	Stateful	0.63 $\pm 0.10$	62,330 $\pm 2,488$	2,308,000 $\pm 35,183$
	<b>SKILL.state</b>	<b>0.78 <math>\pm 0.08</math></b>	<b>2,545 <math>\pm 471</math></b>	<b>90,200 <math>\pm 2,792</math></b>

## Appendix D. Additional Results

### .1 Software Repository State Recovery Experiment 3 using Gemini-3-Flash.

### .2 Extended Budget-Matched Scaling across Horizons

Table 11 presents the full multi-horizon scaling results for the budget-matched control configurations on the Warehouse environment ( $T \in \{10, 25, 50, 100\}$ , Gemini-3-Flash), evaluated across 5 environmental seeds. While all budget-matched runtimes perform comparably at short horizons ( $T = 10$ ), sliding-window truncation and statistical perplexity compression (LLMLingua) experience catastrophic accuracy collapse as horizon length increases, whereas SKILL.state maintains robust performance across all horizons.

Table 7: Gemma-4-31b-it Warehouse Scaling.

Horizon	Runtime	Score $\pm$ SD	Avg Prompt $\pm$ SD	Total Tokens $\pm$ SD
10 Steps	Prompt	0.90 $\pm$ 3.1%	3,145 $\pm$ 242	9,092 $\pm$ 1,231
	Memory	0.85 $\pm$ 4.2%	2,611 $\pm$ 138	7,330 $\pm$ 838
	Stateful	0.90 $\pm$ 2.8%	3,191 $\pm$ 149	9,144 $\pm$ 518
	SKILL.state	0.98 $\pm$ 1.5%	<b>2,116 <math>\pm</math> 212</b>	<b>6,814 <math>\pm</math> 875</b>
25 Steps	Prompt	0.64 $\pm$ 5.4%	5,720 $\pm$ 182	41,697 $\pm$ 1,610
	Memory	0.72 $\pm$ 4.8%	3,990 $\pm$ 362	28,933 $\pm$ 412
	Stateful	0.76 $\pm$ 4.1%	5,714 $\pm$ 282	39,314 $\pm$ 585
	SKILL.state	0.84 $\pm$ 3.6%	<b>2,080 <math>\pm</math> 114</b>	<b>16,302 <math>\pm</math> 2,190</b>
50 Steps	Prompt	0.31 $\pm$ 6.2%	10,809 $\pm$ 352	151,845 $\pm$ 2,150
	Memory	0.41 $\pm$ 5.8%	7,217 $\pm$ 316	114,113 $\pm$ 1,620
	Stateful	0.55 $\pm$ 5.1%	11,083 $\pm$ 262	155,164 $\pm$ 2,210
	SKILL.state	0.68 $\pm$ 3.9%	<b>2,113 <math>\pm</math> 176</b>	<b>33,762 <math>\pm</math> 1,385</b>
100 Steps	Prompt	0.21 $\pm$ 4.7%	27,686 $\pm$ 412	923,164 $\pm$ 12,400
	Memory	0.24 $\pm$ 4.2%	18,537 $\pm$ 229	701,954 $\pm$ 9,850
	Stateful	0.42 $\pm$ 4.5%	20,210 $\pm$ 822	557,968 $\pm$ 8,100
	SKILL.state	0.42 $\pm$ 4.1%	<b>2,105 <math>\pm</math> 216</b>	<b>65,480 <math>\pm</math> 1,258</b>

Table 8: Qwen 3-8b-it Warehouse Scaling.

Horizon	Runtime	Score $\pm$ SD	Avg Prompt $\pm$ SD	Total Tokens $\pm$ SD
10 Steps	Prompt	0.84 $\pm$ 3.8%	3,150 $\pm$ 245	9,150 $\pm$ 1,250
	Memory	0.80 $\pm$ 4.5%	2,640 $\pm$ 145	7,420 $\pm$ 860
	Stateful	0.84 $\pm$ 3.4%	3,210 $\pm$ 155	9,210 $\pm$ 540
	SKILL.state	0.94 $\pm$ 2.1%	<b>2,120 <math>\pm</math> 215</b>	<b>6,920 <math>\pm</math> 890</b>
25 Steps	Prompt	0.54 $\pm$ 6.1%	5,790 $\pm$ 195	42,450 $\pm$ 1,680
	Memory	0.62 $\pm$ 5.4%	4,050 $\pm$ 375	29,640 $\pm$ 440
	Stateful	0.66 $\pm$ 4.8%	5,780 $\pm$ 295	39,950 $\pm$ 610
	SKILL.state	0.76 $\pm$ 4.2%	<b>2,088 <math>\pm</math> 120</b>	<b>16,680 <math>\pm</math> 2,240</b>
50 Steps	Prompt	0.24 $\pm$ 6.5%	10,950 $\pm$ 365	154,200 $\pm$ 2,280
	Memory	0.33 $\pm$ 6.1%	7,320 $\pm$ 330	116,400 $\pm$ 1,710
	Stateful	0.44 $\pm$ 5.7%	11,210 $\pm$ 280	158,300 $\pm$ 2,340
	SKILL.state	0.58 $\pm$ 4.5%	<b>2,118 <math>\pm</math> 185</b>	<b>34,510 <math>\pm</math> 1,420</b>
100 Steps	Prompt	0.15 $\pm$ 5.1%	28,150 $\pm$ 430	941,500 $\pm$ 12,800
	Memory	0.18 $\pm$ 4.6%	18,840 $\pm$ 245	718,200 $\pm$ 10,200
	Stateful	0.31 $\pm$ 4.9%	20,580 $\pm$ 850	569,400 $\pm$ 8,450
	SKILL.state	0.34 $\pm$ 4.6%	<b>2,110 <math>\pm</math> 220</b>	<b>66,850 <math>\pm</math> 1,310</b>

Table 9: Software Repository Experiment 2 i.e. Noise Robustness. Evaluation of runtime resilience to irrelevant syslog telemetry (Condition 1) during a 50-step horizon using Gemini-3-Flash.

Noise Level	Runtime	Score
0 Events (Baseline)	Prompt	0.76
	Memory	0.85
	Stateful	0.88
	SKILL.state	0.90
5 Events (Low)	Prompt	0.62
	Memory	0.85
	Stateful	0.86
	SKILL.state	0.88
20 Events (Medium)	Prompt	0.48
	Memory	0.83
	Stateful	0.85
	SKILL.state	0.86
50 Events (High)	Prompt	0.11
	Memory	0.74
	Stateful	0.78
	SKILL.state	0.80

Table 10: Software Repository State Recovery (Env 2, Exp 3). Comparison of hallucination lag (recovery steps) when the repository state is altered via unstructured alerts.

Scenario	Runtime	Success	Recovery Steps
A: Force Push	Prompt	Yes	12
	Memory	Yes	8
	Stateful	Yes	10
	SKILL.state	<b>Yes</b>	<b>0</b>
B: Flaky CI Test	Prompt	Yes	14
	Memory	Yes	9
	Stateful	Yes	11
	SKILL.state	<b>Yes</b>	<b>0</b>
C: PR Closed	All Runtimes	No	N/A

Table 11: Extended Budget-Matched Scaling on Warehouse across Horizons (Gemini-3-Flash, Target Budget  $\sim 1,800$  tokens, Score  $\pm$  SD across 5 seeds).

Horizon ( $T$ )	SKILL.state	Summary-capped	Truncated (Window)	ReAct + LLMingua	ReAct (Full)
10	<b>1.00</b> $\pm 0.00$	0.92 $\pm 0.03$	0.90 $\pm 0.04$	0.88 $\pm 0.04$	0.90 $\pm 0.02$
25	<b>1.00</b> $\pm 0.00$	0.76 $\pm 0.04$	0.62 $\pm 0.05$	0.60 $\pm 0.05$	0.92 $\pm 0.02$
50	<b>0.96</b> $\pm 0.01$	0.64 $\pm 0.05$	0.35 $\pm 0.06$	0.38 $\pm 0.06$	0.88 $\pm 0.04$
100	<b>0.94</b> $\pm 0.01$	0.52 $\pm 0.05$	0.18 $\pm 0.05$	0.22 $\pm 0.05$	0.84 $\pm 0.07$